

PRAGYA — Plan 2: Prototype Build Plan

1. The Locked Idea

"Don't just find the gap. Find the gap behind the gap."

PRAGYA is a teacher-centric AI diagnostic tool for one FLN vertical: **subtraction and regrouping**, Class 3–5. It doesn't tell a teacher "this child is weak in subtraction" — it finds *why*, verifies the cause, groups students who share it, recommends one activity per group, and re-tests to check the gap actually closed.

Locked pipeline: Assess → Trace → Diagnose → Verify → Group → Intervene → Reassess.

Governing principle: rules decide *what* happened; AI only investigates *why* — and never chooses the final diagnosis alone. The teacher always has the last word.

How This Plan Was Built

This is the locked, final concept — no more changes to the core idea. Two things from the earlier PRAGYA Brutal Audit are kept on top of it, because they're correctness fixes, not scope changes:

- **No invented confidence.** A verified gap is reported as a count ("2 of 3 micro-diagnostic answers point to regrouping"), never an unexplained percentage.
- **Gap-closure percentages are real.** The before/after numbers in the reassessment ARE shown as percentages — but only because they're computed from actual response counts across the group (e.g. 3 of 10 correct before, 8 of 10 after → 30% → 80%), never guessed. This is the honest version of the locked idea's own "32% → 81%" beat.

Everything else below follows the locked idea exactly: one vertical, the 7-stage pipeline, the 83 – 47 demo, and the three locked roles.

2. Scope

In scope (P0, locked)	Later waves (P1, ours — slack time only)	Stretch (P2 — only if P1 is fully done)
Class/student profile: create a class, add students	Addition with carry	Voice-based assessment (spoken digit, teacher confirms before saving)

In scope (P0, locked)	Later waves (P1, ours — slack time only)	Stretch (P2 — only if P1 is fully done)
Short predefined FLN assessment: 5 items, subtraction with regrouping, Class 3–5	AI explanation text + Hindi translation	Read-aloud, WhatsApp share, question builder, installable app, login
One shared 5-item assessment run for a class of 30	Printable blank assessment sheet (paper administration)	
4 candidate causes: calculation slip, place value, regrouping, word problem	Printable results sheet (class run + activity)	
Teacher-entered response — tap the answer given, or type a number	Parent report per child, English/Hindi, printable	
Rule-based diagnosis (aggregated across all 5 items) + 3-question verification + teacher override	Landing page, saved/exported data, language toggle, mobile polish, photo/OCR input (teacher confirms every reading)	

3. The Assessment & Locked Example

The short predefined FLN assessment is 5 items, all 2-digit subtraction requiring regrouping — the locked example is item 3. Every wrong option a teacher can select is pre-tagged to one of the 4 causes, so scoring is a lookup, never a guess.

Item	Type	Problem	Correct	Regrouping-signature wrong answer
1	Computation	62 – 35	27	37
2	Word problem	Rani had 91 rupees. She spent 58 on books. How much is left?	33	43 (or 149, if the wrong operation is chosen — tagged word-problem difficulty)
3	Computation (locked example)	83 – 47	36	46

Item	Type	Problem	Correct	Regrouping-signature wrong answer
4	Computation	54 – 26	28	38
5	Computation	71 – 39	32	42

Every regrouping-signature value above comes from the same rule as the locked example (Section 6): borrow correctly into the ones column, but keep the tens digit instead of reducing it. Each item's place-value-confusion and calculation-slip wrong options are Abhay's to finalize by hand tonight, using that same rule as the template — never invented on the spot.

Why 46 specifically, for item 3: the child borrows correctly into the ones column ($13 - 7 = 6$) but forgets to reduce the tens digit (keeps 8 instead of 7), so $8 \text{ tens} + 6 \text{ ones} = 46$. This is a precise, real misconception — not a random wrong number — and it's what the engine's rule for "regrouping misconception" must reproduce exactly.

4. User Flow

flowchart LR

A[ASSESS
enter question + answer] --> B[TRACE
identify the error type]

B --> C[DIAGNOSE
rank candidate causes]

C --> D[VERIFY
3 micro-diagnostics]

D --> E[GROUP
cluster the class by verified cause]

E --> F[INTERVENE
one activity per group]

F --> G[REASSESS
measure gap closure]

5. Candidate Causes & the 3 Micro-Diagnostics

Cause	What it looks like	Micro-diagnostic that tests it
Place-value confusion	Treats each digit as a separate number rather than tens and ones	"What is the value of 8 in 83?" — wrong answer (e.g. "8") confirms this
Regrouping misconception	Borrows into the ones column but forgets to reduce the tens digit ($83 - 47 \rightarrow 46$)	"Show how you would regroup 83." — can't demonstrate the borrow correctly confirms this

Cause	What it looks like	Micro-diagnostic that tests it
Calculation slip	Understands place value and regrouping, but makes an arithmetic error	Passes both other checks, but the original answer was still wrong
Word-problem difficulty	Understands the arithmetic in isolation, but picks the wrong operation from a word problem	Only tested when the original item was a word problem, not a bare computation
"Can you subtract 7 from 3 directly?"	A support question, not itself a verdict	Confirms the child sees why borrowing is needed at all — feeds the regrouping check

Verify's decision rule, in order:

1. Fails the place-value check → **Verified: Place-Value Misconception** (fix the foundation first)
2. Passes place-value, fails the regroup-demonstration check → **Verified: Regrouping Misconception**
3. Passes both, but the original answer was still wrong → **Verified: Calculation Slip**
4. Original item was a word problem and the wrong operation was chosen → **Verified: Word-Problem Difficulty**

PRAGYA never skips straight to a verdict from the first wrong answer alone — all applicable micro-diagnostics run before anything is verified, exactly as the locked idea specifies.

6. Diagnosis Logic (Deterministic Core)

```
// Step 1 – each wrong answer across the 5-item assessment is evidence for
// one cause,
// looked up from that item's pre-tagged options (never computed live from
// the number alone)
function likelyCandidates(responses: { itemId: string; cause: Cause }[]) {
  const hits: Record<string, number> = {};
  for (const r of responses) if (r.cause !== 'on-track') hits[r.cause] =
(hits[r.cause] ?? 0) + 1;
  return Object.entries(hits)
    .sort((a, b) => b[1] - a[1] || causeOrder(a[0]) - causeOrder(b[0])) //
ties: foundation first
    .map(([cause]) => cause as Cause);
}

// Step 2 – Verify always runs for the top candidate, regardless of how many
// items pointed to it
function verify(placeValueOk: boolean, canRegroupOk: boolean, wasWrong:
boolean) {
  if (!placeValueOk) return 'place-value';
  if (!canRegroupOk) return 'regrouping';
  return wasWrong ? 'calculation' : 'on-track';
}
```

For the locked student, missing items 2, 3 and 5 all carry the regrouping tag, so "regrouping" is already the top likely candidate with 3 of 5 items as evidence before verification even starts — verification then confirms it with real evidence rather than guessing from one number alone. This is stronger than judging from a single wrong answer, which is exactly why the assessment is 5 items, not 1.

7. Classroom Group Stage (Locked Numbers, Reproduced)

Thirty students take the 5-item assessment. Each student's Group is their top verified cause across all 5 items — the seed data is built to reproduce the locked idea's own example exactly:

Result	Students	Verified cause
Correct (36)	12	On track
Answers 46	10	Regrouping misconception

Result	Students	Verified cause
Answers show digit confusion (e.g. treats 83 as 8 and 3 separately)	5	Place-value misconception
Answer is wrong but place-value and regrouping checks both pass	2	Calculation slip
Answer suggests the wrong operation was chosen for context	1	Word-problem difficulty

18 students show a subtraction difficulty, split 10 / 5 / 2 / 1 — exactly the locked idea's own breakdown.

8. Intervention Library (One Per Group)

Cause	Activity (~10 min)	Materials
Regrouping misconception	Break-a-bundle: show 83 as 8 bundles + 3; untie one bundle to get 7 bundles + 13; physically cross out the 8 and write 7 before subtracting	Sticks, rubber bands
Place-value misconception	Place-value mat: build the number with tens bundles and loose sticks, say it aloud as "8 tens and 3 ones"	Sticks, place-value mat
Calculation slip	Timed, low-pressure practice with the same problem type, checked against a partner	Worksheet, pair work
Word-problem difficulty	Act it out: children physically act "gave away / fewer" vs. "more/altogether" before choosing the operation	Story cards

9. Reassessment & Gap Closure

Each group gets 3 fresh problems of the same type (a different subtraction pair each time, still requiring regrouping, generated from the rule in section 6). Before/after is the plain correct-count percentage across the group — never a diagnostic confidence number.

Worked example, Regrouping group (10 students): before the activity, 3 of 10 solve the 3 new problems correctly (30%). After the activity, 8 of 10 do (80%). That's the demo's gap-closure beat — grounded in real counts, matching the spirit of the locked idea's "32% → 81%" without inventing a number.

10. Where AI Is Used, and Where It Isn't

Task	Deterministic or AI-assisted
Scoring, likely-candidate ranking, the Verify decision rule, gap-closure percentage	Deterministic — the rules in Section 6
Which cause is finally verified	Deterministic, but the teacher can Accept / Reject / Change it
Plain-language explanation of the verified cause (P1)	AI-assisted
Hindi translation of the activity (P1)	AI-assisted
Reading a photographed answer sheet (P1)	AI-assisted, but the teacher confirms every reading before it's used

Prompt 1 — teacher explanation (P1)

You help a primary school teacher in India. Explain the verified cause below in 2-3 short, simple sentences and give one quick thing to check in class. Use ONLY the cause provided. Do not suggest any other cause, any learning disability, or any label for the child. Maximum 60 words.
Cause: {name} – {meaning}
Evidence: {which micro-diagnostics failed}

Prompt 2 — Hindi activity (P1)

Translate this classroom activity into simple Hindi (Devanagari) that a Class 3 teacher can read aloud.
Keep all numbers as digits. Do not add, remove, or change any steps. Return only the translation.
{activity text}

The API key lives only in `.env.local`, read by the server route `app/api/explain/route.ts`. It is never sent to the browser and never committed — the repo is public.

11. Data Model (lib/types.ts — The Contract)

```

type Cause = 'place-value' | 'regrouping' | 'calculation' | 'word-problem' |
'on-track';

interface ClassProfile { id: string; name: string; grade: string }
interface Student { id: string; classId: string; name: string }
interface Problem { id: string; a: number; b: number; isWordProblem?: boolean
}
interface ObservedResponse { studentId: string; problemId: string; given:
number; cause: Cause }
interface MicroDiagnosticResult { placeValueOk: boolean; canRegroupOk:
boolean }
interface Diagnosis {
  studentId: string;
  likely: Cause[]; // ranked from all 5 items, most-evidenced
  first
  verified?: Cause; // set once Verify runs
  evidence: string[]; // e.g. ["3 of 5 items: regrouping",
"failed: show regrouping"]
  teacherDecision?: 'accepted' | 'rejected' | Cause; // Change overrides
with a new Cause
}
interface Group { cause: Cause; studentIds: string[]; activityId: string }
interface ReassessmentResult { cause: Cause; correctBefore: number;
correctAfter: number; total: number }
interface ParentReportData { student: Student; cause: Cause; activity:
string; language: 'en' | 'hi' }

```

12. Architecture & Stack

```

flowchart LR
  UI[Next.js pages<br/>React state + localStorage] -->
  ENG[lib/engine.ts<br/>pure functions]
  ENG --> DATA[(data/*.json<br/>causes, activities, seed)]
  UI --> API[/api/explain/]
  API --> LLM[LLM provider]
  API -.fallback.-> CACHE[(data/cache.json)]

```

Layer	Choice	Why
Build tools	Antigravity (all three, unlimited, vibe-coding every module)	Speed; typing is no longer the bottleneck

Layer	Choice	Why
	Claude Code (Abhinav only, limited) for the shared contract + review passes	Only he has it; used for cross-module quality, not a single owned feature
App	Next.js (App Router) + TypeScript	One project, one deploy
UI	Tailwind + shadcn/ui	Polished screens fast
Logic	<code>lib/engine.ts</code> , runs in the browser	No backend to break; works offline
Data	JSON files + React state → localStorage	No database; survives refresh
AI (P1)	One LLM API key, separate from Antigravity/Claude Code	2 small calls, both cached
Hosting	Vercel, auto-deploy from the repo	Live link all day

13. Repo Structure

```

pragya/
├─ app/
│   ├─ class/create/page.tsx      # Class/student profile      (Ankit)
│   ├─ assess/page.tsx           # 5-item assess screen        (Ankit)
│   └─ verify/[id]/page.tsx      # Verify + verdict           (Ankit, on
Abhay's engine)
│   ├─ class/page.tsx            # Class run + Group view      (Abhinav)
│   ├─ intervene/page.tsx        # Intervention screen          (Abhinav)
│   ├─ reassess/page.tsx         # Reassess + gap closure      (Abhinav)
│   ├─ print/assessment/page.tsx # Printable blank sheet (P1)  (Ankit)
│   ├─ print/results/page.tsx    # Printable results (P1)      (Ankit)
│   ├─ report/[id]/page.tsx      # Parent report (P1)          (Abhinav)
│   └─ api/explain/route.ts      # LLM route + fallback (P1)  (Abhinav)
├─ components/                  # Shared UI                    (Ankit)
├─ lib/
│   ├─ types.ts                 # Contract                    (Abhinav)
│   ├─ engine.ts                # Diagnosis + verify logic    (Abhay)
│   ├─ causeGraph.ts            # The 4 causes + rules       (Abhay)
│   └─ seed.ts                  # Class + 30-student generator (Abhay)
├─ data/
│   ├─ items.json               # The 5 assessment items      (Abhay)
│   ├─ activities.json          # Intervention library         (Abhay)
│   └─ cache.json               # Pre-generated AI text (P1)  (Abhinav)
├─ .env.example                 # Key name only, no value
└─ README.md

```

14. Seed Data Design

The generator creates a class of 30 students, then produces exactly the Section 7 split across the 5-item assessment for a fixed random seed, so every demo run is identical: 12 students answer 36 (correct), 10 answer 46 (regrouping signature), 5 show a place-value-confused entry, 2 show a plain calculation slip, 1 answers as if solving a word problem incorrectly. Keep one extra student unassessed for the live, on-stage walkthrough.

15. Demo Script — 90 Seconds (Locked)

Time	Screen
0:00–0:15	Student response: $83 - 47 = 46$ ❌
0:15–0:30	PRAGYA: subtraction error detected; candidate causes listed — calculation, place value, regrouping, word problem
0:30–0:55	Micro-diagnostic: the 3 questions run live
0:55–1:05	🔍 Verified Root Gap: Regrouping — the "PRAGYA moment"
1:05–1:20	10 students in this class share the same root gap — targeted intervention generated
1:20–1:30	Regrouping: 30% → 80- real counts, not a guess

Closing line: "PRAGYA doesn't just tell a teacher that a student is struggling. It helps explain why — and what to do next." Extend to 3:00 with: "A wrong answer is a symptom. PRAGYA finds the underlying skill gap, verifies it, and turns that diagnosis into action."

16. Mentor Script

Don't ask "what should we build?" Say instead:

"We've scoped PRAGYA to one FLN vertical — subtraction and regrouping. Our core pipeline is Assess → Trace → Diagnose → Verify → Group → Intervene → Reassess. We have six hours. Can you help us identify the one part that could undermine the credibility of our diagnosis?"

Use the mentor for architecture validation, edge cases, demo credibility and technical shortcuts — not for deciding scope, which is already locked.

17. Judge Q&A

Question	Answer
"Isn't this just Mindspark?"	Mindspark is student-facing adaptive software needing device time per child. PRAGYA is a teacher-operated check on one shared problem, and every verdict shows its evidence and is confirmable by the teacher.
"Where does your confidence come from?"	We don't show made-up percentages for the diagnosis — we show which checks passed or failed. The only percentage we show is gap-closure, computed from real reassessment counts.
"What if the AI is wrong?"	The verified cause is never AI-decided — it's a rule applied to which micro-diagnostics passed or failed. The teacher can also accept, reject or change it.
"Why only one vertical?"	Proving one diagnosis end-to-end, verified and reassessed, is stronger than a shallow pass over many skills. Addition, literacy and more are the next phase, not tomorrow's demo.
"What about children's data?"	Names stay in the teacher's browser and are never sent to any API; a real deployment needs verifiable parental consent under the DPDP Act, handled through the school.
"What if the API goes down?"	Only the P1 explanation/Hindi text uses an API, and it falls back to cached text. The diagnosis itself needs no network.
"Who would use or pay for this?"	Teachers use it; state FLN cells under Samagra Shiksha are the realistic buyer, as a layer on top of assessments they already run.
"What's your weakest point?"	One vertical, proven on a seeded class — real classroom validation is exactly what a one-school pilot would do next.

18. Risks & Fallbacks

Risk	Fallback
First integration (12:30) not end-to-end	All 3 stop new work and fix it together; every P1 item is cancelled
An agent breaks working code	Revert to the last commit; write a smaller instruction
Micro-diagnostic answers come out ambiguous for a seeded student	Fix the seed data, not the rule — the rule in Section 6 is the one thing that must stay simple and correct
LLM (P1) slow, erroring, or Wi-Fi down	Serve <code>data/cache.json</code> — the demo never waits on an API
One vertical looks thin to a judge	Have the addition-with-carry extension (P1, item 10) ready to show if asked "is that it?"
15:15 — anything still broken	Hide it. Never demo a half-working screen

18a. Definition of Done

- ☐ A teacher can create a class and add students before running anything
- ☐ The locked example (item 3: $83 - 47 = 46$) runs end-to-end within the 5-item assessment: Assess → Diagnose → Verify → Verified: Regrouping
- ☐ Class run reproduces the 12 / 10 / 5 / 2 / 1 split
- ☐ Group → Intervene → Reassess shows a real, count-based gap-closure percentage
- ☐ Teacher Accept / Reject / Change works on the verified gap
- ☐ Visual gap report (bottleneck bars) matches the class split
- ☐ If time allowed: printable blank assessment sheet, printable results, and a parent report all open cleanly
- ☐ Demo run timed under 1:30, twice, extendable to 3:00
- ☐ Public repo with small commits from 10:00 onward
- ☐ No API key anywhere in the repo or its history
- ☐ README: problem, how it works, how to run, screenshots, who built what, that AI agents were used as the organisers allowed
- ☐ Backup screen recording on two laptops
- ☐ Submission form filled before 16:30

19. Agent Prompt Pack

Save these as text tonight; paste them tomorrow. Every prompt starts from `AGENTS.md`. All are written for AntigraVity, since every module is agent-built there — Abhinav can rerun the engine and review prompts in Claude Code as a second check.

AGENTS.md (first commit)

```
# PRAGYA — instructions for every agent
Product: a teacher-run diagnostic for one FLN vertical — subtraction with
regrouping, Class 3-5. A wrong answer is evidence; rules decide which of 4
causes (place-value, regrouping, calculation, word-problem) is verified,
using 3 micro-diagnostics. AI (if used) only explains and translates — it
never decides the diagnosis.
Stack: Next.js App Router, TypeScript strict, Tailwind, shadcn/ui. No
database: React state + localStorage.
Contract: lib/types.ts — never change it unless the task says so.
Folders: app/ and components/ = UI. lib/engine.ts and lib/causeGraph.ts = the
diagnosis rules. data/ = content.
Only edit the folders named in the task. No new dependencies without asking.
Never read, print or commit .env files. Never send student names to any API.
UI never re-implements the diagnosis rule — it calls lib/engine.ts.
Done = works in the browser, no TypeScript errors.
```

Engine (Abhay, AntigraVity, 10:15)

```
Read AGENTS.md and lib/types.ts. Implement lib/causeGraph.ts and
lib/engine.ts exactly as specified below: scoring all 5 assessment responses
against data/items.json's tagged options, the likely-candidate ranking
aggregated across those 5 (most-evidenced cause first, ties broken by
foundation-first order), and the Verify decision rule (place-value check
first, then regrouping, then calculation slip, then word-problem). Show me
your plan before writing code. Only modify lib/.
[paste Plan 2 sections 5 and 6]
```

Assess screen (Ankit, AntigraVity, 10:15)

```
Read AGENTS.md. Build two screens. app/class/create/page.tsx: teacher names a
class, adds students (or loads the seeded demo class). app/assess/page.tsx:
runs the 5-item assessment in data/items.json for one student, one item at a
time (tap the answer given, or type a number). Use lib/mock.ts for now. Only
modify app/class, app/assess and components/.
Done when: [paste feature 2's "done when" from Plan 1]
```

Dashboard + Verify (Abhinav, AntigraVity, 10:15)

Read AGENTS.md. Build the Verify screen (3 micro-diagnostic questions from Section 5, one at a time) and the verdict view (Section 5's decision rule output, shown as a count of checks passed, with Accept/Reject/Change). Call lib/engine.ts once it exists; use lib/mock.ts until then. Only modify app/verify, app/class, components/.

Done when: [paste features 4 and 5's "done when" from Plan 1]

Review pass (Abhinav, Claude Code, 12:30 and 15:15)

Review the whole repo for: API keys or secrets in code or git history; student names sent to any API; TypeScript errors; broken imports; any place the UI re-implements the diagnosis rule instead of calling lib/engine.ts. List issues by severity. Fix only the critical ones, after I approve.